



Unidad de Ingeniería en Computación

Campus Tecnológico Local San Carlos

Spoticry

Curso: Lenguajes de Programación

Profesor: Oscar Viquez

Autores:

Kevin Said Reyes Quesada
Juan Pablo Quesada Caballero

28 de abril de 2026

Tablas de contenidos

DESCRIPCIÓN DEL PROBLEMA Y OBJETIVOS	3
OBJETIVOS.....	4
OBJETIVO GENERAL	4
OBJETIVOS ESPECÍFICOS.....	4
ANÁLISIS TÉCNICO DE LA SOLUCIÓN IMPLEMENTADA	5
ARQUITECTURA GENERAL.....	5
MANEJO DE ARCHIVOS DE MÚSICA	5
ADMINISTRACIÓN EN MEMORIA DE LA LISTA DE CANCIONES (SERVIDOR).....	7
LOS TRES CRITERIOS DE BÚSQUEDA	8
PROGRAMACIÓN FUNCIONAL EN EL MANEJO DE PLAYLISTS.....	9
MANEJO DE CONFLICTOS Y EXCEPCIONES	10
CONFLICTO DE ELIMINACIÓN DE CANCIÓN EN REPRODUCCIÓN.....	10
FALLOS DURANTE LA SUBIDA DE ARCHIVOS	11
EXCEPCIONES DE RED (SUPABASE NO DISPONIBLE)	11
FALLO DE CONEXIÓN AL CERRAR EL NAVEGADOR	12
PLAYLIST NO ENCONTRADA	12
HALLAZGOS SOBRE SOCKETS, PARALELISMO Y SINCRONIZACIÓN	13
EL PROGRAMA SIN SINCRONIZACIÓN	13
EL PROGRAMA CON SINCRONIZACIÓN (IMPLEMENTACIÓN ACTUAL).....	13
ACTIX-WEB Y EL MODELO DE CONCURRENCIA DE TOKIO	14
RESULTADOS OBTENIDOS	15
SERVIDOR	15
CLIENTE	15
CONCLUSIONES Y RECOMENDACIONES	16
CONCLUSIONES	16
RECOMENDACIONES.....	17
MANUAL DE USO	18

Descripción del Problema y Objetivos

El proyecto consiste en diseñar e implementar una aplicación de reproducción y administración de música distribuida en dos módulos bien diferenciados: un servidor que administra el catálogo de canciones, playlists y el estado de reproducción, y un cliente que expone una interfaz gráfica al usuario final para buscar, organizar y reproducir música.

La particularidad del problema radica en que los dos módulos deben comunicarse a través de una red, operar de forma concurrente (el servidor puede atender múltiples clientes en paralelo), y respetar restricciones de paradigma: el servidor se implementa en Rust con énfasis en programación imperativa eficiente y concurrente, y el manejo de playlists debe seguir principios de programación funcional (inmutabilidad, map, filter, fold, closures).

Objetivos

Objetivo General

Implementar una aplicación cliente-servidor de administración y reproducción de música que integre técnicas de programación concurrente en el lado del servidor (Rust) y programación funcional en el manejo de listas de reproducción, con una interfaz de usuario web reactiva en el lado del cliente (React).

Objetivos Específicos

1. Implementar un servidor en Rust que exponga una API REST capaz de atender solicitudes concurrentes de múltiples clientes.
2. Habilitar búsqueda de canciones bajo al menos tres criterios técnicamente distintos.
3. Implementar el manejo de playlists en Rust utilizando exclusivamente transformaciones funcionales (sin mutabilidad).
4. Desarrollar un cliente web en React con interfaz gráfica que permita reproducir canciones usando un buffer local para poder adelantar y retroceder.
5. Implementar una interfaz de texto (CLI) en el servidor para la administración directa de canciones.
6. Garantizar que una canción no pueda eliminarse mientras esté siendo reproducida activamente por algún cliente.
7. Permitir la carga de archivos de audio e imagen directamente desde la computadora del usuario, con almacenamiento automático en Supabase Storage.

Análisis Técnico de la Solución Implementada

Arquitectura General

La aplicación sigue una arquitectura cliente-servidor de tres capas. Supabase actúa como capa de persistencia, proveyendo tanto la base de datos como el almacenamiento de los archivos .mp2 y .png en el storage. El servidor Rust construido con Actix-Web actúa como capa de lógica de negocio e intermediario. El cliente React/Vite es la capa de presentación.

Decisión de protocolo — HTTP/JSON en lugar de TCP puro: El enunciado original sugería sockets TCP puros. Sin embargo, el cliente es una aplicación web que corre dentro de un navegador. Los navegadores, por razones de seguridad del sandbox, no permiten abrir conexiones TCP arbitrarias desde JavaScript. La única forma de comunicación de red disponible en el browser es HTTP o WebSockets. Por esta razón se optó por HTTP/REST con JSON sobre Actix-Web, lo que además simplifica la interfaz de comunicación, facilita el debugging con herramientas estándar y sigue siendo concurrente: Actix-Web sobre Tokio atiende múltiples solicitudes en paralelo sin bloquearse.

Manejo de Archivos de Música

El almacenamiento de archivos de audio e imágenes se realiza en Supabase Storage, dentro de un único bucket llamado spoticry organizado en dos subcarpetas: songs/ para archivos MP3 y covers/ para portadas de álbumes.

Flujo de subida (implementación final): El servidor Rust actúa como intermediario en el proceso de carga. El cliente envía los archivos como multipart/form-data junto con los metadatos textuales de la canción. El flujo es el siguiente:

8. Cliente React envía POST /admin/songs con multipart/form-data (campos: name, artist, genre, album; archivos: song MP3, image JPG/PNG).
9. Servidor Rust sube el MP3 a Supabase Storage en spoticry/songs/{timestamp}-{nombre}.mp3.
10. Servidor sube la imagen a Supabase Storage en spoticry/covers/{timestamp}-{nombre}.jpg.
11. Servidor construye las URLs públicas de ambos archivos.
12. Servidor inserta la entrada en la tabla songs de la base de datos con las URLs generadas.

El servidor usa la SECRET_API_KEY (llave de rol de servicio) para autenticarse contra Supabase Storage, lo que garantiza que las credenciales sensibles nunca sean expuestas al navegador del cliente.

Para evitar colisiones de nombres entre archivos, cada archivo recibe un nombre único generado en el servidor combinando el timestamp en milisegundos con el nombre original sanitizado:

```
let ts = SystemTime::now()
    .duration_since(UNIX_EPOCH)
    .unwrap_or_default()
    .as_millis();
let unique = format!("{}", ts, sanitize_filename(&filename));
```

La función `sanitize_filename` reemplaza cualquier carácter que no sea alfanumérico, punto, guión o guión bajo por un guión bajo, evitando problemas con caracteres especiales en las URLs de Storage.

Una vez que Supabase Storage acepta el archivo, la URL pública sigue el patrón fijo: `https://{proyecto}.supabase.co/storage/v1/object/public/spoticry/{carpeta}/{nombre}`. Esta URL es la que queda almacenada en la base de datos y la que el cliente React usa directamente para reproducir el audio, sin pasar nuevamente por el servidor Rust.

El servidor está configurado con un límite de payload de 50 MB (`web::PayloadConfig::default().limit(50 * 1024 * 1024)`), suficiente para cualquier canción MP3 de calidad estándar (una canción de 5 minutos a 320 kbps ocupa aproximadamente 12 MB).

En lugar de solicitar URLs manualmente, la página de administración presenta zonas de arrastre (drag & drop) para cada archivo. Al soltar o seleccionar un MP3, se muestra el nombre y tamaño del archivo. Al seleccionar una imagen, se muestra una miniatura de previsualización generada localmente con `URL.createObjectURL()`, que se libera de memoria al cambiar el archivo o limpiar el formulario.

Una vez que la canción existe en la base de datos con su URL pública, el cliente React descarga el archivo de audio completo mediante `fetch(track.src)` al iniciar una reproducción. El `audioBuffer` completo se mantiene en memoria del navegador, lo que permite hacer seek sin necesidad de volver a descargar el archivo.

Administración en Memoria de la Lista de Canciones (Servidor)

Las canciones se almacenan de forma persistente en la base de datos de Supabase. El servidor Rust no mantiene una copia en memoria del catálogo completo; cada solicitud de búsqueda o listado dispara una consulta HTTP a la API REST de Supabase en tiempo real.

```
let mut url = format!("{}/rest/v1/songs?select=*", base_url);
if let Some(nombre) = &query.nombre {
    url.push_str(&format!("&name=ilike.*{}", nombre));
}
if let Some(artista) = &query.artista {
    url.push_str(&format!("&artist=ilike.*{}", artista));
}
if let Some(genero) = &query.genero {
    url.push_str(&format!("&genre=eq.{}", genero));
}
```

Lo que sí se administra en memoria del servidor es el conjunto de IDs de canciones actualmente en reproducción:

```
type PlayingNow = Arc<Mutex<HashSet<String>>>>;
```

Arc (Atomic Reference Counting) permite que múltiples threads/tareas asincrónicas compartan el mismo HashSet sin copiar los datos. Mutex garantiza acceso exclusivo para lecturas y escrituras, evitando condiciones de carrera.

Las playlists se persisten también en Supabase (tabla playlists con columna song_ids de tipo array). El servidor las lee de la BD, aplica transformaciones funcionales en memoria, y retorna el resultado sin mutar la BD excepto cuando el usuario explícitamente agrega o elimina canciones de una playlist.

Los Tres Criterios de Búsqueda

El servidor implementa tres criterios técnicamente distintos:

- Nombre (?nombre=): Utiliza el operador ilike.*valor* de Supabase. Busca cualquier canción cuyo nombre contenga el texto ingresado en cualquier posición, sin distinguir mayúsculas/minúsculas.
- Artista (?artista=): Igual mecánica que nombre pero aplicada al campo artista. Técnicamente el mismo operador pero semánticamente diferente: el usuario busca por autor, no por título.
- Género (?genero=): Utiliza el operador eq.valor. A diferencia de los anteriores, exige coincidencia exacta. Es un filtro por enum, no una búsqueda de texto libre.

Los tres pueden combinarse en la misma solicitud, permitiendo búsquedas compuestas como "canciones de género Rock cuyo nombre contenga 'star'".

Programación Funcional en el Manejo de Playlists

Se exige que el manejo de playlists use exclusivamente transformaciones funcionales: map, filter, fold, closures, sin mutabilidad. Se identifican tres puntos clave en el código del servidor:

a) Eliminar canción de playlist — filter con closure:

```
let nuevos_ids: Vec<String> = song_ids
    .into_iter()
    .filter(|id| id != &song_id)
    .collect();
```

La lista original song_ids se consume y se produce nuevos_ids como colección nueva. No hay mutación en el lugar.

b) Ordenamiento de canciones — sort_by con closures:

```
match query.orden.as_deref() {
    Some("artista") => canciones_ordenadas.sort_by(|a, b|
a.artist.cmp(&b.artist)),
    Some("album")   => canciones_ordenadas.sort_by(|a, b|
a.album.cmp(&b.album)),
    Some("reciente") => canciones_ordenadas.reverse(),
    _ => {}
}
```

Cada closure |a, b| ... es una función pura de comparación que no modifica a ni b.

c) Guardia de reproducción — eliminación condicional:

```
let playing_set = playing.lock().unwrap();
if playing_set.contains(&song_id) {
    return HttpResponse::Conflict()
        .body("No se puede eliminar: la canción está siendo
reproducida");
}
drop(playing_set);
```

Manejo de Conflictos y Excepciones

Conflicto de Eliminación de Canción en Reproducción

El escenario más crítico de conflicto es intentar eliminar una canción que un cliente está escuchando. El flujo es:

- i. Inicio de reproducción: el cliente React llama POST /stream/start/{id}. El servidor inserta el ID en el HashSet protegido por Mutex.
- ii. Intento de eliminación: si el CLI o un cliente llama DELETE /admin/songs/{id}, el handler adquiere el lock, verifica si el ID está en el set y retorna HTTP 409 Conflict si es así.
- iii. Fin de reproducción: el cliente llama POST /stream/stop/{id}. El ID se elimina del set.

```
let playing_set = playing.lock().unwrap();
if playing_set.contains(&song_id) {
    return HttpResponse::Conflict()
        .body("No se puede eliminar: la canción está siendo
reproducida");
}
drop(playing_set); // libera antes del request HTTP a Supabase
```

Nótese que el lock se libera antes de hacer la solicitud de borrado a Supabase para no bloquear el Mutex durante una operación de red que podría tardar cientos de milisegundos.

Fallos durante la Subida de Archivos

El endpoint POST /admin/songs realiza tres operaciones externas en secuencia (upload MP3, upload imagen, inserción en DB). Si cualquiera falla, el handler retorna inmediatamente con un mensaje de error descriptivo:

```
Ok(res) => {
  let status = res.status();
  let body = res.text().await.unwrap_or_default();
  eprintln!("Error subiendo MP3: {} - {}", status, body);
  return HttpResponse::InternalServerError()
    .body("Error al subir el archivo de audio");
}
```

Si el upload del MP3 tiene éxito pero el de la imagen falla, el archivo MP3 ya subido queda huérfano en Storage (sin una entrada correspondiente en la DB). Para el alcance del proyecto esto es aceptable, pero se señala como punto de mejora en la sección de recomendaciones.

Excepciones de Red (Supabase no disponible)

Cada solicitud al servidor Supabase está envuelta en un match sobre el Result del request HTTP y otro match sobre el parseo JSON:

```
match client.get(&url)...send().await {
  Ok(res) => match res.json::<Vec<Song>>().await {
    Ok(songs) => HttpResponse::Ok().json(songs),
    Err(e) => {
      eprintln!("Error al parsear: {}", e);
      HttpResponse::InternalServerError().body("Error al parsear
respuesta")
    }
  },
  Err(_) => HttpResponse::InternalServerError().body("Error al
conectar con Supabase"),
}
```

El cliente React en api.js captura respuestas HTTP no exitosas y lanza Error con el mensaje del servidor, que los componentes muestran en pantalla.

Fallo de Conexión al Cerrar el Navegador

Para el escenario donde el usuario cierra abruptamente el navegador, se usa `navigator.sendBeacon`:

```
window.addEventListener('beforeunload', () => {  
  if (streamingTrackIdRef.current) {  
    navigator.sendBeacon(`/stream/stop/${trackId}`)  
  }  
})
```

`sendBeacon` envía una solicitud POST fire-and-forget que el navegador garantiza entregar incluso durante el cierre de la pestaña, resolviendo la race condition de "usuario cerró el tab y la canción quedó marcada como en reproducción indefinidamente".

Playlist No Encontrada

Si el usuario navega directamente a una URL de playlist que ya fue borrada, el cliente detecta que el `getPlaylist(id)` devuelve null y renderiza un estado de error con botón para volver a la biblioteca, sin crash de la aplicación.

Hallazgos sobre Sockets, Paralelismo y Sincronización

El Programa sin Sincronización

Sin el Mutex protegiendo el `HashSet<String>`, múltiples handlers HTTP corriendo en diferentes workers de Actix-Web podrían producir las siguientes situaciones problemáticas:

- Race condition de eliminación: un handler de delete podría pasar el chequeo `contains()` justo antes de que otro handler haga el `insert()` para la misma canción. El resultado sería una canción borrada mientras está en reproducción.
- Corrupción del `HashSet`: sin sincronización, dos writers concurrentes sobre la misma estructura de datos pueden producir estados completamente inválidos a nivel de memoria.

En Rust, el compilador impide este escenario directamente: no es posible compartir una referencia mutable entre threads sin `Mutex`, `RwLock`, o primitivas similares. Intentarlo produce un error de compilación, no un bug en tiempo de ejecución. Esta es una de las garantías más importantes del lenguaje para sistemas concurrentes.

El Programa con Sincronización (implementación actual)

Con `Arc<Mutex<HashSet<String>>>`:

- Garantía de exclusión mutua: solo un handler a la vez puede tener el lock del `Mutex`. Si dos clientes intentan iniciar/detener reproducción de la misma canción simultáneamente, uno espera.
- Lock de granularidad fina: el lock se adquiere solo para la operación sobre el `HashSet`, no para toda la solicitud HTTP.

- Drop explícito antes de I/O: el patrón `drop(playing_set)` antes de llamar a Supabase es clave. Si el lock se retuviera durante el request de red, todos los demás handlers que necesiten verificar `playing_now` quedarían bloqueados por la latencia de Supabase. Con el drop temprano, el lock se libera en microsegundos.

Actix-Web y el Modelo de Concurrency de Tokio

Actix-Web usa un modelo de actores asíncronos sobre Tokio. Cada `async fn` handler se suspende en operaciones de I/O (`.await`) sin bloquear el thread del runtime. Esto permite que un único thread maneje cientos de conexiones concurrentes, cada una esperando su respuesta de Supabase, sin necesidad de crear un thread por conexión.

El CLI usa `spawn_blocking` justamente porque leer de `stdin` sí bloquea, y bloquearlo dentro del runtime Tokio paralizaría todos los handlers HTTP en ese worker. Del mismo modo, la acumulación de bytes del multipart dentro del handler `add_song` es asíncrona: los chunks se van recibiendo con `.await` sin bloquear otros requests.

Resultados Obtenidos

La aplicación final implementa de forma funcional los siguientes módulos:

Servidor

- GET /songs con filtros por nombre, artista y género.
- POST /admin/songs con recepción de archivos multipart/form-data, subida automática a Supabase Storage y creación de entrada en la base de datos.
- DELETE /admin/songs/{id} para eliminación con guardia de reproducción activa.
- POST /stream/start/{id} y POST /stream/stop/{id} para registro de reproducción.
- POST /playlist, GET /playlist, DELETE /playlists/{id} para CRUD de playlists.
- POST /playlists/{id}/songs/{song_id} y DELETE /playlists/{id}/songs/{song_id} para gestión de canciones en playlists.
- GET /playlists/{id}/songs?orden=artista|album|reciente para listado con ordenamiento funcional.
- CLI de texto con comandos listar, agregar y eliminar <id>.

Cliente

- Interfaz adaptada para escritorio y móvil.
- Reproducción con buffer completo (Web Audio API): play, pause, seek, next, previous.
- Control de volumen con persistencia en localStorage.
- Modos repeat (off/all/one) y shuffle.
- Búsqueda de canciones por nombre o artista.
- Biblioteca de playlists con creación, eliminación y visualización.
- Agregar/eliminar canciones de playlists con picker visual.
- Ordenamiento de canciones dentro de playlists (por artista, álbum, reciente).
- Sincronización automática de playlists y canciones cada 4-5 segundos.
- Página de administración con zonas de drag & drop para subir MP3 e imágenes con previsualización.

Conclusiones y Recomendaciones

Conclusiones

- i. Usar HTTP/REST en lugar de TCP puro fue la decisión correcta. Dado que el cliente es una aplicación web en el navegador, no era posible abrir sockets TCP directamente desde JavaScript. HTTP permitió una comunicación clara y funcional entre los módulos, sin sacrificar la capacidad de atender múltiples clientes al mismo tiempo gracias al runtime asíncronico de Rust.
- ii. Rust demostró ser un lenguaje muy adecuado para manejar concurrencia de forma segura. El compilador obliga a proteger los datos compartidos entre hilos, lo que eliminó desde el principio problemas típicos como condiciones de carrera. Si el código compila, esas garantías están dadas.
- iii. Mantener la llave de Supabase en el servidor fue importante. Al centralizar la subida de archivos en el servidor Rust, las credenciales sensibles nunca llegan al navegador del usuario, lo cual es una práctica de seguridad básica que el diseño respeta correctamente.
- iv. El buffer completo de audio en el cliente funciona bien para este proyecto, pero consume memoria considerable con canciones largas. Para una aplicación real con un catálogo extenso esto sería un problema que habría que resolver con streaming progresivo.

Recomendaciones

- i. Streaming de audio en lugar de descarga completa. Actualmente el cliente descarga el MP3 entero antes de reproducirlo. En una aplicación real, lo correcto sería reproducir mientras se descarga, reduciendo el tiempo de espera y el uso de memoria.
- ii. Agregar autenticación. Hoy cualquier persona que conozca la API puede agregar o eliminar canciones. Para una versión real se necesitaría al menos un sistema de login que distinga entre usuarios normales y administradores.
- iii. Manejar archivos huérfanos. Si durante la subida de una canción el MP3 se sube correctamente pero la imagen falla, el archivo de audio queda guardado en Storage sin ninguna canción que lo referencie. Sería necesario implementar una limpieza automática de esos archivos.
- iv. Agregar pruebas automatizadas. El proyecto no cuenta con tests, lo que hace difícil verificar que todo sigue funcionando correctamente al hacer cambios. Implementar pruebas básicas de los endpoints sería el primer paso hacia una versión más robusta y mantenible.

Manual de Uso

Como ejecutar en terminal

1. Para poder activar el server se tiene que abrir el proyecto y despues ingresar la carpeta y ejecutar el comando de “cargo run”.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  POSTGRESQL QUERY RESULTS

● saidrq@Kevins-MacBook-Air Spoticry % cd server
○ saidrq@Kevins-MacBook-Air server % cargo run
```

2. Para poder activar el cliente se tiene que abrir el proyecto y despues ingresar la carpeta y ejecutar el comando de “npm run dev”

```
● saidrq@Kevins-MacBook-Air Spoticry % cd client
○ saidrq@Kevins-MacBook-Air client % npm run dev
```

3. Cuando el server esta corriendo se muestra el uso de este en la terminal, con listar se pueden ver todas las canciones y sus ID

```
○ saidrq@Kevins-MacBook-Air server % cargo run
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.34s
   Running `target/debug/Spoticry`
🎵 Spoticry corriendo en http://localhost:8080
— Consola Spoticry —
Comandos: listar | agregar | eliminar <id>

listar
Easy On Me | Adele | Pop | ID: 5
LOVE YOU LESS | Joji | Pop | ID: 6
```

4. Como agregar una canción, es importante notar que desde las terminal se tiene que ingresar el link del .mp3 y el link del cover de la canción, ya que desde aquí no se puede seleccionar archivos desde su pc, solo los url desde una base de datos.

```
agregar
Nombre de la canción:
Run
Artista:
Joji
Género:
Pop
URL del archivo de audio:
<AQUÍ TIENE QUE IR UNA URL DEL .mp3>
URL de la imagen:
<AQUÍ TIENE QUE IR UNA URL DE LA imagen>
Álbum:
Sanctuary
Canción agregada
```

5. Para eliminar solo se escribe el comando y el numero del ID de la canción a eliminar

```
eliminar <se coloca el id de la canción que se desea eliminar que se ve lista>
nnect
```

Ln 207, Col 1 S